Table des matières

Cours 04 — Extraire d'une base ou d'une API 1
Ce que vous allez faire 1
Avant de commencer 1
Étape 1 — Lire une API page par page · Ingestion 2
Étape 2 — Écrire `meteo.py` · Ingestion 3
Étape 3 — Tirer juin · Ingestion 4
Étape 4 — Comparer l'API à l'instantané livré · Ingestion 5
Étape 5 — Versionner · Ingestion 6
À retenir 7

Cours 04 — Extraire d'une base ou d'une API

Cours 4/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
	✓ ici	

Ce que vous allez faire

Jusqu'ici, les sources arrivaient en fichiers, déposés dans `sources/`. Bien des sources ne livrent rien : on va les chercher, dans une base ou par une API, et elles répondent par morceaux. Le café n'a pas de base de données, mais sa météo vient d'une API publique, celle d'Environnement et Changement climatique Canada (ECCC). Vous écrivez `meteo.py`, qui tire juin 2025 de l'API page par page et le dépose dans le lac, puis vous le comparez à l'instantané que le mois figé livre déjà.

l'API d'ECCC, dix jours par page → `meteo.py` → `plateforme/lac/bronze/meteo/`, une ligne JSON par jour → la comparaison avec l'instantané livré

Le code complet est dans `Codes_04_DE.md`, à ouvrir après avoir essayé. Ce cours demande Internet.

Avant de commencer

© 2026 iAlexMG. Tous droits réservés.

Une seule fenêtre de l'Anaconda Prompt sert à ce cours, placée dans `Projet_Brulerie` : le notebook s'y ouvre à l'étape 4 et s'y ferme avant la dernière.

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
dir /b sources\meteo
REM Attendu : LISEZMOI.md et montreal_quotidien.csv.
```

`montreal_quotidien.csv` est l'instantané : les 30 jours de juin à la station de l'aéroport de Montréal, MONTREAL INTL A, tirés de la même API le 2026-09-14. Le dictionnaire des sources en donne les colonnes, section 5.

Étape 1 — Lire une API page par page · Ingestion

Une API web répond à une adresse : on lui demande une collection, avec des paramètres, et elle rend du JSON. Celle d'ECCC suit la norme OGC API Features : la collection `climate-daily` porte un jour d'une station par entité. Une réponse n'en rend qu'un nombre limité, une page, et dit où trouver la suivante.

Dans l'adresse	**Ce qu'il demande**
`CLIMATE_IDENTIFIER=7025251`	la station : MONTREAL INTL A
`datetime=2025-06-01 00:00:00/2025-06-30 00:00:00`	la période, bornes comprises
`sortby=LOCAL_DATE`	les jours dans l'ordre
`limit=10`	dix jours par page
`f=json`	la réponse en JSON

Dans la réponse	**Ce qu'il dit**
`features`	les entités de la page ; chacune porte ses valeurs sous `properties`
`numberReturned`	combien la page en rend
`numberMatched`	combien la demande en trouve en tout
`links`	des liens, chacun avec son `rel` ; celui dont `rel` vaut `next` mène à la page suivante

© 2026 iAlexMG. Tous droits réservés.

On lit donc la première page, puis celle que son lien `next` désigne, et ainsi de suite, jusqu'à une page qui n'en annonce plus. Dix jours par page, c'est peu pour un mois : c'est voulu, pour voir les pages passer.

Étape 2 — Écrire `meteo.py` · Ingestion

Le script n'emploie que la bibliothèque standard de Python. Il ne réessaie pas après une coupure : le cours 05 dit ce qu'un pipeline fait d'un échec.

Pour remplir	**L'outil**
l'adresse et ses paramètres	`urllib.parse.urlencode(PARAMETRES)` rend `f=json&CLIMATE_IDENTIFIER=…`, les espaces et les `/` encodés
appeler l'adresse	`urllib.request.urlopen(url, timeout=60)`, dans un `with`, qui ferme la connexion
lire la réponse	`json.load(reponse)` rend un dictionnaire Python
le lien suivant, ou rien	`next((… for lien in page["links"] if …), None)` rend le premier lien qui convient, ou `None`
écrire le fichier	`SORTIE.write_text(texte, encoding="utf-8")` le réécrit en entier

Le dépôt se fait dans le lac, à côté du bronze des tickets, une ligne JSON par jour, telle que l'API la rend. Comme le bronze, le fichier se réécrit en entier à chaque passage : relancer ne double rien.

2.1 Créer le fichier, dans la première fenêtre.

```
notepad 01_de\meteo.py
```

Accepter la création, puis écrire le code à partir de ce pseudocode, les trous `____` compris :

```
# 01_de/meteo.py : la météo de juin, tirée de l'API d'ECCC page par page, déposée dans le lac
import json
import urllib.parse
import urllib.request
from pathlib import Path

API = "https://api.weather.gc.ca/collections/climate-daily/items"
# 1. l'aéroport de Montréal, du 1er au 30 juin, dix jours par page
PARAMETRES = {"f": "json", "CLIMATE_IDENTIFIER": "7025251", "sortby": "LOCAL_DATE",
              "datetime": "2025-06-01 00:00:00/2025-06-30 00:00:00", "limit": ____}
SORTIE = Path("plateforme/lac/bronze/meteo/montreal_2025-06.jsonl")
```


© 2026 iAlexMG. Tous droits réservés.

```
def tirer(url):
    jours = []
    while url:
        # 2. une page : ses jours, puis le lien vers la suivante, s'il y en a une
        with urllib.request.urlopen(url, timeout=60) as reponse:
            page = json.load(reponse)
        jours += [jour["properties"] for jour in page["____"]]
        print(f"page : {page['numberReturned']} jours, {len(jours)} sur {page['numberMatched']}")
        url = next((lien["href"] for lien in page["links"] if lien["rel"] == "____"), None)
    return jours

if __name__ == "__main__":
    jours = tirer(f"{API}?{urllib.parse.urlencode(PARAMETRES)}")
    # 3. une ligne JSON par jour, le fichier réécrit en entier : relancer ne double rien
    SORTIE.parent.mkdir(parents=True, exist_ok=True)
    ____
    print(f"{len(jours)} jours dans {SORTIE.as_posix()}")
```

Le trou de l'étape 3 tient en une ligne : `"".join(…)` colle les jours, chacun rendu en texte par `json.dumps(jour, ensure_ascii=False)` et suivi d'un saut de ligne, `"\n"`. Enregistrer, puis fermer le Bloc-notes.

Étape 3 — Tirer juin · Ingestion

3.1 Lancer le script. Il faut Internet.

```
python 01_de\meteo.py
REM Attendu : page : 10 jours, 10 sur 30 ; puis 20 sur 30 et 30 sur 30 ; puis
REM page : 0 jours, 30 sur 30, et 30 jours dans plateforme/lac/bronze/meteo/montreal_2025-06.jsonl.
```

Quatre pages pour 30 jours : la troisième annonçait encore une suite, et la quatrième, vide, n'en annonce plus. Une API ne dit pas toujours qu'elle a fini avant la page qui le montre ; c'est le lien `next` absent qui arrête la boucle, pas un compte.

Si le script s'arrête sur	**La cause, et le geste**
`urllib.error.URLError: <urlopen error [Errno 11001] getaddrinfo failed>`	pas d'Internet, ou le nom du serveur mal écrit : vérifier la connexion et la ligne `API`
`urllib.error.HTTPError: HTTP Error 404: NOT FOUND`	le serveur répond, mais la collection n'existe pas : relire la fin de la ligne `API`, `climate-daily/items`
`KeyError: 'feature'`	un nom de clé mal écrit : `features`, au pluriel, comme dans la réponse

3.2 Relancer le script, pour prouver qu'il se rejoue sans rien doubler.

© 2026 iAlexMG. Tous droits réservés.

```
python 01_de\meteo.py
REM Attendu : les mêmes quatre pages, puis 30 jours dans plateforme/lac/bronze/meteo/montreal_2025-06.jsonl.
dir /b plateforme\lac\bronze
REM Attendu : meteo et tickets.
```

Le fichier a été réécrit, pas complété : il garde ses 30 jours.

Étape 4 — Comparer l'API à l'instantané livré · Ingestion

L'instantané a été tiré de la même API, cinq semaines plus tôt. Les deux devraient dire la même chose ; une source qu'on va chercher se compare quand même à ce qu'on a déjà, parce qu'une API peut corriger ses valeurs après coup. La comparaison a un piège, qu'on provoque exprès.

4.1 Ouvrir le notebook de DuckDB, depuis l'Anaconda Prompt.

```
duckdb -ui
```

Créer un notebook et le renommer « Cours 04 ». Il travaille en mémoire, comme aux cours 01 et 02 : `read_json` lit le fichier de l'API comme `read_csv` lit l'instantané.

4.2 Compter les jours tirés, et lire leurs types. Une requête par cellule.

```
-- 1. les jours du fichier, le premier et le dernier
SELECT COUNT(*) AS jours, MIN(LOCAL_DATE) AS premier, MAX(LOCAL_DATE) AS dernier
FROM read_json('plateforme/lac/bronze/meteo/*.jsonl');
-- Attendu : 30 jours, du 2025-06-01 00:00:00 au 2025-06-30 00:00:00.

DESCRIBE SELECT LOCAL_DATE, MEAN_TEMPERATURE, MAX_TEMPERATURE, TOTAL_PRECIPITATION, SNOW_ON_GROUND
FROM read_json('plateforme/lac/bronze/meteo/*.jsonl');
-- Attendu : LOCAL_DATE en TIMESTAMP, les trois mesures en DOUBLE, SNOW_ON_GROUND en JSON.
```

Le JSON porte ses types : un nombre y est un nombre. `SNOW_ON_GROUND`, vide tout le mois, n'en a pas : DuckDB n'a rien d'où deviner son type, et le garde en JSON. L'API rend aussi plus de colonnes que l'instantané n'en a gardé : 33 contre 20, les rafales et l'humidité en plus.

4.3 Erreur voulue : comparer l'API et l'instantané comme du texte. `all_varchar = true` lit l'instantané tout en texte, comme la `RAW` le ferait, et `CAST(… AS VARCHAR)` ramène les nombres de l'API au texte.

```
-- 1. par jour, la même valeur de chaque côté, comparée comme du texte : un écart compte 1
WITH api AS (SELECT * FROM read_json('plateforme/lac/bronze/meteo/*.jsonl')),
instantane AS (SELECT * FROM read_csv('sources/meteo/montreal_quotidien.csv', all_varchar = true))
SELECT COUNT(*) AS jours,
       SUM(CASE WHEN CAST(a.MAX_TEMPERATURE AS VARCHAR) = i.MAX_TEMPERATURE THEN 0 ELSE 1 END) AS t_max,
       SUM(CASE WHEN CAST(a.TOTAL_PRECIPITATION AS VARCHAR) = i.TOTAL_PRECIPITATION THEN 0 ELSE 1 END) AS precipitations
FROM api AS a
```


© 2026 iAlexMG. Tous droits réservés.

```
INNER JOIN instantane AS i ON i.LOCAL_DATE = CAST(CAST(a.LOCAL_DATE AS DATE) AS VARCHAR);
-- Attendu : 30 jours, 3 écarts de température maximale, 22 de précipitations.
```

`WITH` nomme un résultat intermédiaire, comme au cours 02. Vingt-cinq écarts, sur une API et un instantané qui viennent de la même source.

4.4 Regarder les trois écarts de température.

```
-- 1. les jours où les deux textes diffèrent, et ce que porte chaque côté
SELECT CAST(a.LOCAL_DATE AS DATE) AS jour, a.MAX_TEMPERATURE AS api, i.MAX_TEMPERATURE AS instantane
FROM read_json('plateforme/lac/bronze/meteo/*.jsonl') AS a
INNER JOIN read_csv('sources/meteo/montreal_quotidien.csv', all_varchar = true) AS i
    ON i.LOCAL_DATE = CAST(CAST(a.LOCAL_DATE AS DATE) AS VARCHAR)
WHERE CAST(a.MAX_TEMPERATURE AS VARCHAR) <> i.MAX_TEMPERATURE
ORDER BY jour;
-- Attendu : 3 lignes, 2025-06-02, 20.0 contre 20 ; 2025-06-11, 24.0 contre 24 ; 2025-06-28, 27.0 contre 27.
```

`20.0` et `20` sont deux textes différents, et un seul nombre. Les 22 écarts de précipitations sont du même ordre : `0.0` contre `0`.

4.5 Refaire la comparaison dans le bon type : en nombres. Sans `all_varchar`, `read_csv` lit les mesures de l'instantané en `DOUBLE`, et les dates en `DATE`.

```
-- 1. la même comparaison, chaque valeur dans le type de ce qu'elle représente
WITH api AS (SELECT * FROM read_json('plateforme/lac/bronze/meteo/*.jsonl')),
instantane AS (SELECT * FROM read_csv('sources/meteo/montreal_quotidien.csv'))
SELECT COUNT(*) AS jours,
       SUM(CASE WHEN a.MAX_TEMPERATURE = i.MAX_TEMPERATURE THEN 0 ELSE 1 END) AS t_max,
       SUM(CASE WHEN a.TOTAL_PRECIPITATION = i.TOTAL_PRECIPITATION THEN 0 ELSE 1 END) AS precipitations
FROM api AS a
INNER JOIN instantane AS i ON i.LOCAL_DATE = CAST(a.LOCAL_DATE AS ____);
-- Attendu : 30 jours, 0 et 0.
```

Pas un seul écart : l'API n'a rien corrigé en juin depuis l'instantané. Deux valeurs se comparent dans le type de ce qu'elles représentent ; dans le mauvais, une comparaison invente des écarts, ou en cache de vrais.

4.6 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans l'Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, toujours dans `Projet_Brulerie`.

Étape 5 — Versionner · Ingestion

5.1 Valider le script ; le fichier de l'API, dans le lac, reste hors du dépôt.

© 2026 iAlexMG. Tous droits réservés.

```
git -C 01_de add meteo.py
git -C 01_de commit -m "Cours 04 : la météo de juin, tirée de l'API page par page"
REM Attendu : 1 file changed.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

- Une API se lit page par page : on suit le lien de la page suivante jusqu'à ce qu'il n'y en ait plus, et l'on compte ce qu'on a reçu contre ce qu'elle annonce.
- Ce qu'on va chercher se dépose tel que la source le rend, et se réécrit en entier : relancer ne double rien.
- Deux valeurs se comparent dans le type de ce qu'elles représentent : en texte, `20.0` et `20` diffèrent ; en nombres, non.

© 2026 iAlexMG. Tous droits réservés.